

EXPRESSIONS

CS1910
Computer Science I

CODE EXAMPLES

- https://colab.research.google.com/github/UPEI-CS/CSI910-Students/blob/main/lectures/04-Expressions/CSI910_Expressions_Workbook.ipynb

EXPRESSIONS

- Combinations of literal values, variables, operators and function calls that the python interpreter evaluates to produce a single value
 - Primarily compute and return results
- Examples of Types of expressions
 - Arithmetic expressions (+ - * / // **, ...)
 - Comparison expressions (> < >= <= ==, ...)
 - Logical expressions (and or not)
 - Function call expression (abs(), len(), ...)

ARITHMETIC EXPRESSIONS

- Follows mathematical operator precedence (BEDMAS)
 - Left to right evaluation
- Supports: + - * / // % **
- Be mindful of the return type for the expression
 - Generally, if ints and floats are both involved then the result is “elevated” to float
- String concatenation is supported with +

SELF CHECK

- What do the following arithmetic expressions evaluate to?
(Be mindful of your return types)

```
int("5" + "5") + int("5") + int("5")
```

```
3 // 5 * 12345
```

```
"hello" + "world"
```

```
77 / 10 + 77 % 10
```

```
"hello" + str(8/2)
```

```
5 + 4.2 + 0.8
```

```
8 / 2
```

```
7.0 % 2
```

← Python supports modulus with floating point numbers

ASSIGNMENT OPERATORS

- Recall from last day that variables are assigned a value using =
 - Example: `a = 29`
- Select Other Assignment Operators:
 - `+=` Addition Assignment (add and then assign)
 - `-=` Subtraction Assignment
 - `/=` Division Assignment
 - `//=` Floor Division Assignment
 - `*=` Multiplication Assignment
 - `%=` Modulus Assignment

ASSIGNMENT OPERATORS EXAMPLES

```
1 #Addition Assignment Example
2 bank_balance = 1000
3 deposit = 50
4
5 bank_balance += deposit
6 print(bank_balance)
```

```
1 #Concatenation Assignment
2 my_string = "hello"
3 my_string += " world!"
4
5 print(my_string)
```

Predictions on what this outputs?

```
1 x = 121
2 count = 0
3
4 while(x >= 11):
5     x -= 11
6     count += 1
7
8 print(count)
```

More on while loops later,
essentially executes the indented
code until the clause is False

PAIR
PROBLEM
SOLVING /
DISCUSSION

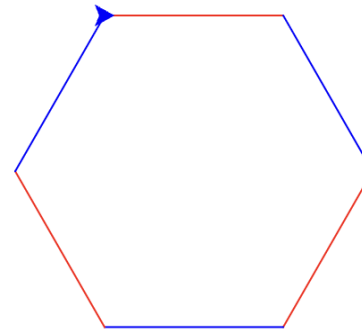
- Problem 1:
- How can we determine if a number is even or odd?

Difficulty Estimate: fairly straight-forward

Note: We aren't using **if** statements just yet

PAIR
PROBLEM
SOLVING /
DISCUSSION

- Problem 2:
- How can we turtle draw a hexagon so that the sides alternate red and blue?



Difficulty Estimate: same as previous

COLORED HEXAGON

- 2 ways to change the Turtle's color:

`color("red")` ← string

`color(r, g, b)` e.g., `color(255, 0, 0)`

3 integers

Integer values between 0 and 255

COLORED TRIANGLE

This code "works" but it hasn't taken full advantage of our computational thinking skills

There are a few variables (abstraction) but no patterns (we repeat the same code in a bunch of places) - this is not good for code maintenance nor readability

Can we take advantage of patterns?

Change Color to BLUE

```
5 #draw a hexagon with
6 #alternating colors
7 length = 100
8 angle = 360/6
9
10 color(255, 0, 0)
11 forward(length)
12 right(angle)
13 |
14 color(0,0,255)
15 forward(length)
16 right(angle)
17
18 color(255,0,0)
19 forward(length)
20 right(angle)
21
22 color(0, 0, 255)
23 forward(length)
24 right(angle)
25
26 color(255,0,0)
27 forward(length)
28 right(angle)
29
30 color(0,0,255)
31 forward(length)
32 right(angle)
```

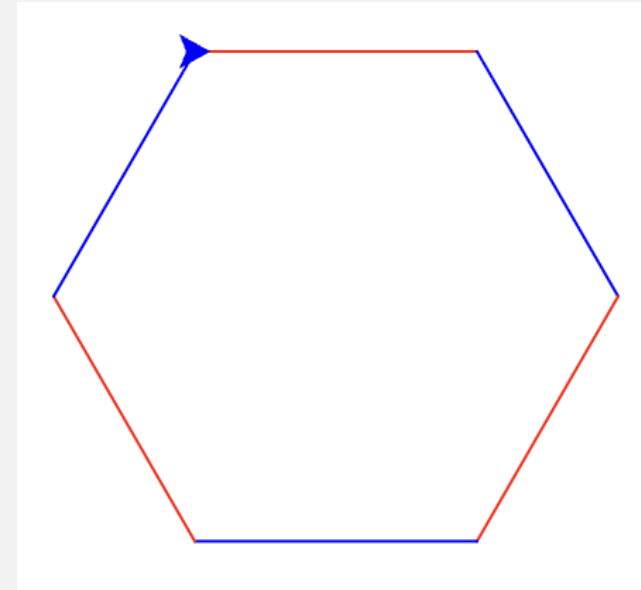
PATTERN RECOGNITION

- From last day we know the pattern for the shape:

```
# some variables to make our solution more robust (abstraction)
FULL_CIRCLE = 360 #ALL_CAPS denotes constants (things that never change)
length = 100
sides = 6
angle = FULL_CIRCLE / sides

#patterns to break down the shape into smaller bits
for side in range(sides):
    forward (length)
    right(angle)

#side starts at 0 and iterates upto but not including sides (6)
#i.e., it goes from 0 to 5 (inclusive)
print(side)
```



QUESTIONS TO ASK

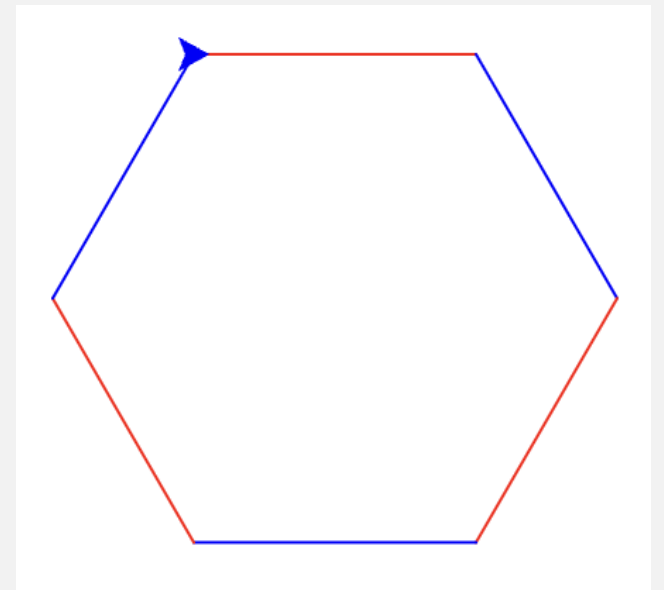
1. What are we trying to achieve?
2. What do we know?
3. What information is missing?
4. Can we break it down into smaller parts?
5. What are the patterns?
6. Can we represent it differently?
7. Have we solved it before?

QUESTIONS TO ASK

- What are we trying to achieve?
 - Draw red, blue, red blue – but actually ... We are trying to track if it is red and blue.
- What do we know?
 - 6 sides, 120 degree turn ... and ... first side is red.
 - We know that red and blue are represented with combinations of 0 and 255.
- Can we break it down into smaller parts?
 - We need to know if the first number r should be 0 or 255. We know g is 0. We need to know if the number b should be 0 or 255.

WHAT ARE THE PATTERNS?

- The colors have a pattern too that we can use.
- Consider that the first side is the top side and call it
 - Side 1 – Red
 - Side 2 – Blue
 - Side 3 – Red ...
- Odd numbered sides are red
- Even numbered sides are blue
- Pattern recognition:
 - Essentially the problem “Decomposes” drawing each side and keeping a count of the side we are on so we can determine if it is even or odd



CAN WE REPRESENT IT DIFFERENTLY?

- “Odd” sides are (255, 0, 0) <- RED
- “Even” sides are (0, 0, 255) <- BLUE
- Can we develop a formula that converts a side number like 2 into (0,0,255)?

HAVE WE SOLVED IT BEFORE?

- We need to detect if something is even or odd. How did we do that?
- Modulo operator helps us with that – but that gives us 0 (even) and 1 (odd)
- We need ...
 - (255,0,0)
 - (0,0,255)
 - (255,0,0)
 - (0,0,255)

HAVE WE SOLVED IT BEFORE?

- Logic: We need the following:
 - If side is odd - 1 remainder - then r is 255 and the rest are 0.
 - Else if side is even – 0 remainder – then b is 255 and the rest are 0. (255,0,0)
- How do we get 0 and 1? We did that before:
 - Count the sides in a loop with a variable – side_count
 - $\text{side_count} \% 2 * 255$ – produces 0 or 255

DECOMPOSITION

- We have identified simple patterns and decomposed our problem into:
 - drawing each side (forward and rotate)
 - while using the count of side to set the color.

COLORED HEXAGON

We did this previously

- Pattern

- The shape is just forward and rotate by $360 / 6$ for each of the 6 sides
- But the color?

A CONSTANT (should never change)
Use ALL_CAPS to denote constants

Pseudocode:

START

```
SET FULL_CIRCLE = 360
SET length = 100
SET sides = 6
SET angle = FULL_CIRCLE / sides
```

Small counting adjustment to
start counting from 0

For side_count = 0 to sides-1:

```
  SET the color = ( ...)
```

```
  forward length
```

```
  rotate right angle
```

There are other ways to do this, but
today we develop a solution using
expressions

SETTING COLORS

```
SET green = 0
For side_count = 0 to sides-1:
    SET red = (side + 1) % 2 * 255
    SET blue = side % 2 * 255

    color(red, green, blue)
    forward length
    rotate right angle
```

COLORED HEXAGON SOLUTION

```
1 # Colored Triangle
2
3 from ColabTurtlePlus.Turtle import *
4
5 clearscreen()
6
7 #draw a triangle with
8 #all sides different colors
9 FULL_CIRCLE=360
10 length = 100
11 sides = 6
12 angle = FULL_CIRCLE/sides
13
14 for side in range(sides):
15     red    = (side+1) % 2 * 255
16     green  = 0
17     blue   = (side) % 2 * 255
18
19     color(red, green, blue)
20     forward(length)
21     right(angle)
```

COMPARISON EXPRESSIONS

Expressions involving a comparison operator
Evaluate to True or False

- > greater than
- < less than
- == equal to
- != not equal to
- >= greater than or equal to
- <= Less than or equal to

What is the output?

```
1 a = 4
2 b = 5
3 c = 4
4
5 print(a == c)
6 print(a != b)
7 print(a >= c)
```

LOGICAL EXPRESSIONS

Truth Table for
logical operators

X	Y	X and Y	X or Y	not(X)	not(Y)
T	T	T	T	F	F
T	F	F	T	F	T
F	T	F	T	T	F
F	F	F	F	T	T

and mimics multiply, while **or** mimics addition

Operator precedence: Parenthesis first, then NOT then AND then OR

MIMIC OF MULTIPLY AND ADD

(KIND OF)

X	Y	X AND Y	$I \times I?$
1	1	1	$1 \times 1 = 1$
1	0	0	$1 \times 0 = 0$
0	1	0	$0 \times 1 = 0$
0	0	0	$0 \times 0 = 0$

X	Y	X OR Y	$I + I?$
1	1	1	$1 + 1 \rightarrow$ but OR caps at 1
1	0	1	$1 + 0 = 1$
0	1	1	$0 + 1 = 1$
0	0	0	$0 + 0 = 0$

TRUE/FALSE IN EXPRESSIONS

BEGIN

SET numerator = 10

SET denominator = 0

improper_fraction = denominator > 0

print(improper_fraction)

END

False



TRUE/FALSE IN EXPRESSIONS

BEGIN

SET numerator = 10

SET denominator = 0

improper_fraction = denominator > 0 and (numerator/denominator) > 1

print(improper_fraction)

END

False

True

- But wait ... it has to evaluate the divide by 0 and will fail – why doesn't it fail when we run it?

SHORT CIRCUITS

- Once a result is known python stops processing the logical expression
- Example: `result = True or False`

Processing left to right the result is known after processing the left operand, no need to proceed the result is already True

- Example: `False and True`

Processing left to right the result is known after processing the left operand, no need to proceed, the result is already False

TRUE/FALSE IN EXPRESSIONS

BEGIN

SET numerator = 10

SET denominator = 0

improper_fraction = denominator > 0 and (numerator/denominator) > 1

print(improper_fraction)

END



If denominator is zero
this expression stops
evaluating



No need to run

SHORT CIRCUITS

- In Python dividing by zero will crash a program (ZeroDivisionError)

```
#short circuiting examples
```

```
numerator = 10  
denomintator = 0
```

```
#oops division by zero  
improper_fraction = numerator/denomintator
```

If test_value is zero this expression stops evaluating

```
test_score = 50  
test_value = 100
```

```
pass_test = test_value > 0 and (test_score / test_value) >= 0.5  
print(pass_test)
```



SELF-CHECK

- What is the result of the following logical expression?

```
1 cold_enough = (False and True) or ( not (True and False) or not True ) or True and False and True and False or False or True
```

SELF-CHECK

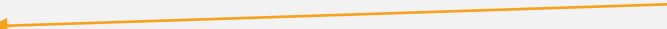
- What is the result of the following logical expression?

```
1 cold_enough = (False and True) or ( not (True and False) or not True ) or True and False and True and False or False or True
```

Hint: use short circuiting

If it helps you can think of the Ors like addition and the And's like multiplication

PYTHON NOTES

- Combining operators can be misleading
 - Consider: `3 < 5 == True`
 - Python translates this to: `3 < 5 and 5 == True`
 - Convenient for:
 - `b_grade = 70 < grade and grade < 80`
 - `70 < grade < 80` 
- Programmers never write `x == True`

Handy compressed format

- Just write `x`

~~• `pass_course = testgrades > 50 == True` ❌~~

- `pass_course = testgrades > 50`

FUNCTION CALL EXPRESSIONS

- All functions in Python return a value
- The function is evaluated, and a value comes back in the place where the function is called
 - Many functions return None (which really means they haven't returned anything)

Examples:

```
sq_root = math.sqrt(9 + 16)
positive_value = abs(any_value)
ph_num_jenny = str(867) + "-" + str(5309)
```

```
max_ = 10
max_ = max(max_, 12)
```

More on functions and built-in functions later

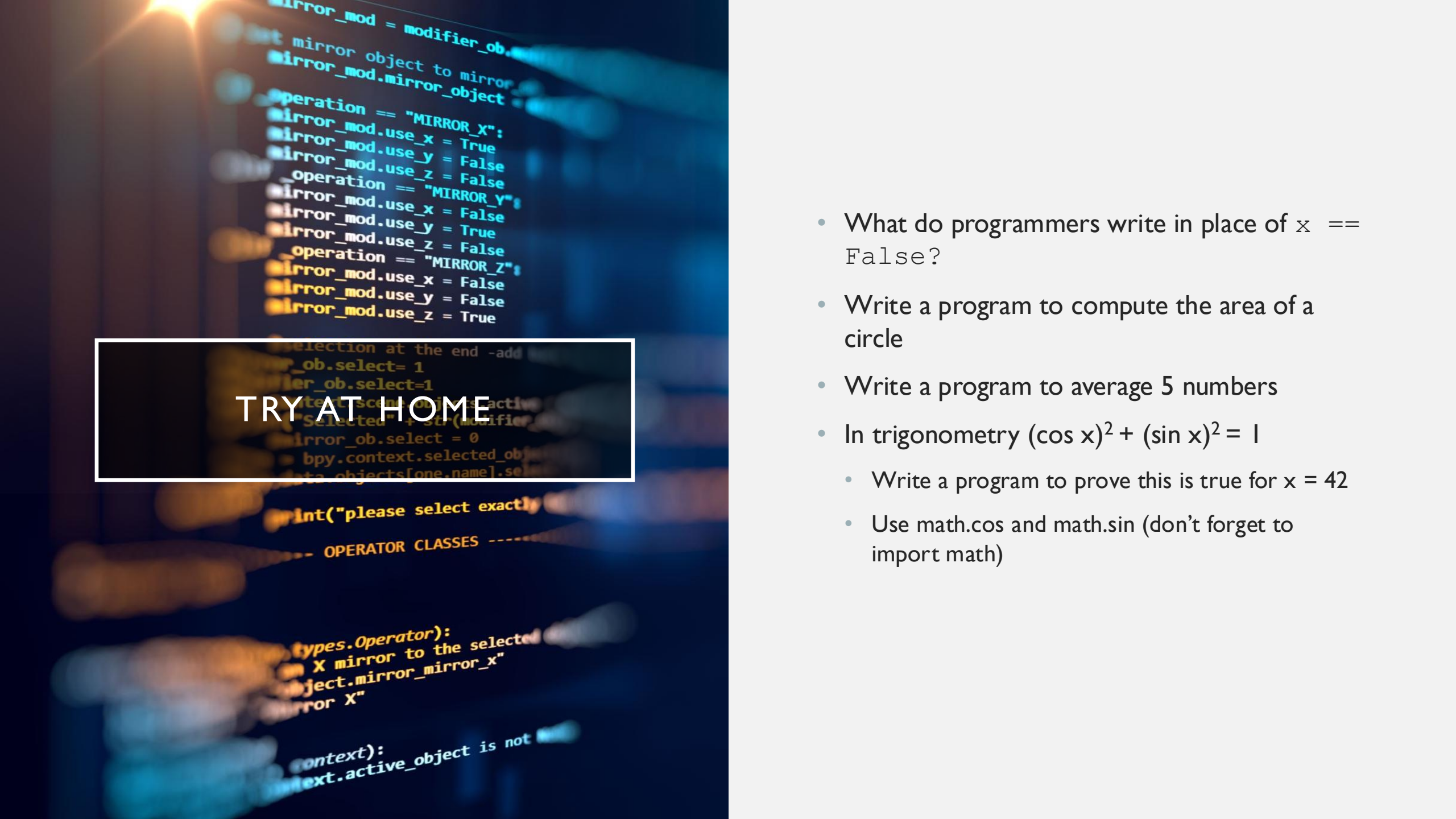
what is the result of the below expression?
value = max(5, min(abs(-20), -6))

SELF-CHECK

Answer: 5

```
# what is the result of the below expression?  
value = max(5, min(abs(-20), -6))
```

SELF-CHECK



TRY AT HOME

- What do programmers write in place of $x == \text{False}$?
- Write a program to compute the area of a circle
- Write a program to average 5 numbers
- In trigonometry $(\cos x)^2 + (\sin x)^2 = 1$
 - Write a program to prove this is true for $x = 42$
 - Use `math.cos` and `math.sin` (don't forget to import `math`)

TRY AT HOME (ADVANCED)

- During the US Open Tennis Tournament all the players descend on the city of NY. They stay in hotels. Tennis players are notoriously superstitious. They will pay a premium if the digits in the room number of their hotel room add up to 1 or they add up to a multi digit number that then sums to 1.
- For example: Room 100 has 3 digits: 1, 0, and 0. These add up to 1 ($1+0+0$). But also consider 532: These add up to 10 but then 10 has digits that sum to 1. So tennis players will pay a premium to stay in room 100 and/or room 532.
- Assume all hotel rooms have 3 digits in their room number. Write a program to output the sum of the digits of the room number as a single digit.
- Your program should output 1 if the room number is 532 and 9 if the room number is 999.

Hint: make use of // and %